Advanced NLP – Exercise 2

Part 1

Netanel Azran

June 2026

Question 1 — Societal Bias Beyond Gender

Paper: Moin Nadeem, Anna Bethke, and Siva Reddy. "StereoSet: Measuring stereotypical bias in pretrained language models." *ACL-IJCNLP 2021.*

StereoSet is a crowdsourced benchmark for measuring how much pretrained language models rely on social stereotypes. It covers four bias categories — race, religion, profession, and gender — with roughly 17,000 items in total. Each item presents a short context followed by three candidate continuations: a *stereotypical* one, an *anti-stereotypical* one, and a semantically *unrelated* one. For example, given "*Arabs are…*," the options might be "violent and dangerous," "peaceful and welcoming," and "made of bricks."

The paper proposes two metrics. The *Stereotype Score* (SS) measures how often the model prefers the stereotypical continuation over the anti-stereotypical one (50 = unbiased). The *Language Model Score* (LMS) checks that the model still prefers meaningful text over nonsense, serving as a sanity check. The two are combined into the *ICAT score*, which rewards models that are both capable and minimally biased. Tested models (BERT, RoBERTa, XLNet, GPT-2) all show clear stereotypical preferences in the race and religion subsets.

Question 2 — Three Efficiency Methods

Method 1: Quantization (Inference)

Quantization reduces the numerical precision of model weights — from float32 down to float16, INT8, or INT4. This can be applied after training (*post-training quantization*) without any retraining, making it very cheap to deploy.

The main saving is *memory*: a 7B-parameter model shrinks from ∼28 GB in float32 to ∼3.5 GB in INT4, which is the difference between needing a data-centre GPU and fitting on a consumer card. Inference speed also improves because lower-precision arithmetic is faster on modern hardware.

The trade-off is a small drop in accuracy, which becomes more noticeable at very aggressive quantization (4-bit). Quantization-aware training recovers most of the loss but requires an extra training phase.

Method 2: LoRA (Training)

LoRA (Hu et al., 2022) fine-tunes a model by freezing all pretrained weights and injecting small low-rank adapter matrices into each transformer layer. Only those adapters are updated; the rest of the model is untouched. After training, the adapters can be merged back into the base weights, so inference cost does not change.

For a 7B model, the number of trainable parameters drops from 7 billion to roughly 20–30 million (under 0.5%), making it practical to fine-tune large models on a single consumer GPU.

The trade-off is expressiveness: because weight updates are constrained to a low-rank subspace, LoRA may fall slightly behind full fine-tuning on some tasks, though in practice the gap is small.

Method 3: Knowledge Distillation (Modeling)

Knowledge distillation (Hinton et al., 2015) trains a small *student* model to mimic a large *teacher* model. Instead of training on hard one-hot labels, the student matches the teacher's full output distribution ("soft" probabilities), which carries richer information about similarity between classes. A well-known NLP example is DistilBERT — a 66M-parameter student trained from BERT-base (110M parameters) that retains ∼97% of BERT's GLUE performance while being 40% smaller and 60% faster.

The trade-off is extra training cost: generating soft labels requires running the large teacher, so distillation takes more compute than training the student from scratch.

Pairwise Combinations

Quantization + LoRA. These two are already combined in practice as **QLoRA** (Dettmers et al., 2023). The frozen base model is quantized to 4-bit (saving the memory needed to hold it), while the LoRA adapters are trained in 16-bit. This makes it possible to fine-tune a 65B model on a single 48 GB GPU — something neither method achieves alone.

Quantization + Knowledge Distillation. A natural pipeline: first distill the large teacher into a smaller student, then quantize the student for deployment. The two savings stack — a student that is 4× smaller than the teacher, then quantized to INT8, ends up using about one-eighth the memory of the original. The only cost is two sequential training phases.

LoRA + Knowledge Distillation. These methods target different problems — LoRA reduces fine-tuning cost while distillation reduces model size — so they are complementary rather than redundant. One practical combination is to distill a large model into a smaller student, then use LoRA to adapt that student cheaply to a downstream task. Useful when both deployment efficiency and training efficiency are required at the same time.